Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-218БВ-25

Студент: Руснак П.Н.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 16.10.26

Москва, 2026

Постановка задачи

Вариант 15.

Цель работы: приобретение практических навыков в управлении потоками в операционной системе и обеспечении синхронизации между потоками.

Задание. Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы. Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы. Необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Условие варианта. Есть колода из 52 карт. Рассчитать экспериментально (методом Монте-Карло) вероятность того, что сверху лежат две одинаковые карты. Количество раундов задаётся ключом программы.

Дополнительные требования преподавателя:

- реализовать две версии алгоритма – последовательную и параллельную – и замерить время выполнения обеих, для параллельной версии – при каждом исследуемом количестве потоков;
- привести таблицу результатов: не менее 4 экспериментов с количеством потоков меньше числа логических процессоров, 1 эксперимент с равным и 3 эксперимента со значительно большим количеством потоков;
- для многопоточности использовать только POSIX Threads (без `threads.h` и `<thread>`), заголовок `stdio.h` не использовать.

Трактовка условия. «Одинаковыми» считаются карты одного достоинства (например, два короля). Теоретическое значение искомой вероятности: какой бы ни была верхняя карта, среди оставшихся 51 карты ровно 3 карты того же достоинства, поэтому $P = 3/51 = 1/17 \approx 0{,}0588$. Это значение используется для проверки правильности программы.

Общий метод и алгоритм решения

Использованные системные вызовы и функции POSIX:

- `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);` – создаёт новый поток, выполняющий функцию start_routine(arg). В Linux реализована через системные вызовы mmap (выделение стека потока) и clone (создание потока в общем адресном пространстве процесса). Возвращает 0 при успехе или код ошибки.
- `int pthread_join(pthread_t thread, void **retval);` – приостанавливает вызывающий поток до завершения указанного потока и освобождает ресурсы завершённого потока; после возврата все записи, сделанные потоком, видны вызывающему потоку.
- `ssize_t write(int fd, const void *buf, size_t count);` – записывает count байт из буфера buf в файловый дескриптор fd; используется для вывода результатов (STDOUT_FILENO) и сообщений об ошибках (STDERR_FILENO) вместо printf.

- `int clock_gettime(clockid_t clockid, struct timespec *tp);` – получает текущее значение часов; используются монотонные часы CLOCK_MONOTONIC, на которые не влияет перевод системного времени.
- `void exit(int status);` – завершает процесс с указанным кодом возврата (1 – при ошибке).

Также используются функции стандартной библиотеки C: malloc и free (выделение и освобождение памяти под массивы потоков), strlen и strcmp (работа со строками), atol (преобразование значений ключей в числа), rand_r (генератор псевдослучайных чисел с состоянием, передаваемым по указателю).

Модель задачи

Колода хранится как массив из 52 целых чисел от 0 до 51; достоинство карты равно остатку от деления числа на 13, поэтому в колоде ровно по 4 карты каждого достоинства. В одном раунде колода полностью перемешивается алгоритмом Фишера–Йетса: для каждой позиции i от 0 до 50 выбирается случайная позиция j из диапазона [i; 51], и карты на позициях i и j меняются местами. Этот алгоритм даёт все 52! перестановок с равной вероятностью. Затем проверяется, совпадают ли достоинства двух верхних карт: deck[0] % 13 == deck[1] % 13.

Оценка вероятности методом Монте-Карло: $\hat{p}$ = (число раундов с парой сверху) / R, где R – общее число раундов. Типичная погрешность оценки равна $\sqrt{p(1-p)/R}$: для $R = 10^7$ она составляет около 0,00007, для $R = 10^8$ – около 0,00002.

Генератор случайных чисел

Используется функция rand_r(&seed), которая хранит состояние генератора в переменной, передаваемой по указателю. В отличие от rand() с общим скрытым состоянием, такой генератор не требует синхронизации между потоками. Каждый поток получает своё зерно (номер потока + 1), поэтому потоки моделируют разные колоды. Последовательная версия и поток 0 получают одинаковое зерно 1, поэтому запуск без ключа -t и с ключом -t 1 даёт одинаковый результат, что используется для проверки корректности.

Последовательная версия

При запуске без ключа -t функция main сама вызывает функцию count_pairs(R, 1), которая проводит все R раундов и возвращает число раундов с парой сверху. Дополнительные потоки не создаются.

Параллельная версия (модель Fork & Join)

При запуске с ключом -t T функция run_parallel делит R раундов между T потоками: каждый получает R / T раундов, а остаток R % T распределяется по одному раунду между первыми потоками, так что сумма долей равна R. Для каждого потока заполняется собственная структура ThreadArgs (число раундов, зерно, поле для результата), после чего поток создаётся вызовом pthread_create и выполняет ту же функцию count_pairs для своей доли раундов. Создаётся ровно T рабочих потоков, поэтому ключ -t задаёт максимальное количество одновременно работающих потоков.

Колода, состояние генератора и счётчик пар являются локальными переменными функции count_pairs и находятся на стеке своего потока, поэтому потоки не изменяют общих данных. Результат поток записывает один раз – в своё поле thread_args[i].hits. Главный поток ожидает все потоки вызовом pthread_join и только после этого суммирует результаты. Поэтому гонка данных невозможна и мьютексы не требуются: синхронизацию обеспечивает pthread_join. Создание и ожидание потоков вынесены в два отдельных цикла, чтобы все потоки работали одновременно.

Замер времени и вывод

Время измеряется вызовами clock_gettime(CLOCK_MONOTONIC) непосредственно до и после вычисления (для параллельной версии – включая создание и ожидание потоков) и выводится в миллисекундах. Поскольку stdio.h не используется, вывод выполняется системным вызовом write, а числа переводятся в текст функцией print_num. Ошибки write, clock_gettime, malloc, pthread_create и pthread_join обрабатываются: в поток ошибок выводится сообщение, и процесс завершается с кодом 1. При неверных или отсутствующих ключах выводится подсказка по использованию.

Формат запуска: `./lab2 -r <раунды>` – последовательная версия; `./lab2 -r <раунды> -t <потоки>` – параллельная версия.

Метрики ускорения и эффективности

Ускорение показывает, во сколько раз параллельная версия выполняется быстрее последовательной:

$$S = T_s / T_p$$

где T_s – время выполнения последовательной версии, мс; T_p – время выполнения параллельной версии на p потоках, мс; p – число потоков. Идеальное значение S = p.

Эффективность показывает, какую долю идеального ускорения удалось получить, то есть насколько полно используется каждый поток:

$$X = S / p$$

где S – ускорение, p – число потоков. Идеальное значение X = 1.

Верхнюю границу ускорения задаёт **закон Амдала**: $S \le 1 / (a + (1 - a) / p)$, где a – доля последовательного (нераспараллеливаемого) кода. В данной задаче последовательная часть (разбор ключей, создание и ожидание потоков, сложение результатов) мала по сравнению с моделированием раундов, поэтому ожидается ускорение, близкое к числу доступных процессорных ядер.

Методика эксперимента

Измерения проводились на ноутбуке с процессором AMD Ryzen 5 7535HS (6 физических ядер, 12 логических процессоров за счёт технологии SMT) в Ubuntu, запущенной в WSL 2; компилятор gcc с оптимизацией -O2. Для каждого числа раундов $R = 10^6, 10^7, 10^8$ измерялись последовательная версия и параллельная версия при p = 1, 2, 4, 6, 8 (меньше числа логических процессоров), p = 12 (равно) и p = 16, 128, 1024 (больше). Замеры выполнялись вспомогательным скриптом bench.sh: перед измерениями выполняется один прогревочный запуск, каждое измерение повторяется 3 раза, в таблицу заносится медиана; ускорение и эффективность вычисляются по приведённым формулам.

Код программы

lab2.c

```
#include <stdlib.h>
#include <string.h>
#include <time.h>

#include <unistd.h>
#include <pthread.h>

#define DECK_SIZE 52

typedef struct {
```

```
	unsigned long rounds; /* сколько раундов провести этому потоку */
	unsigned int seed;    /* зерно генератора случайных чисел потока */
	unsigned long hits;   /* результат: в скольких раундах сверху пара */
} ThreadArgs;

static void print(int fd, const char *s)
{
	if (write(fd, s, strlen(s)) == -1)
		exit(1);
}

static void die(const char *msg)
{
	print(STDERR_FILENO, msg);
	exit(1);
}

static void print_num(unsigned long n, int min_digits)
{
	char buf[32];
	int len = 0;

	do {
		buf[sizeof(buf) - 1 - len++] = '0' + n % 10;
		n /= 10;
	} while (n > 0 || len < min_digits);

	if (write(STDOUT_FILENO, buf + sizeof(buf) - len, len) == -1)
		exit(1);
}

static long long now_us(void)
{
	struct timespec ts;

	if (clock_gettime(CLOCK_MONOTONIC, &ts) == -1)
		die("Ошибка clock_gettime\n");
	return ts.tv_sec * 1000000LL + ts.tv_nsec / 1000;
}

/* Сам алгоритм: rounds раз тасуем колоду и считаем пары сверху */
static unsigned long count_pairs(unsigned long rounds, unsigned int seed)
{
	int deck[DECK_SIZE];
	unsigned long hits = 0;

	for (int i = 0; i < DECK_SIZE; ++i)
		deck[i] = i; /* достоинство карты = deck[i] % 13 */

	for (unsigned long r = 0; r < rounds; ++r) {
		/* Фишер-Йетс: на позицию i ставим случайную карту из i..51 */
		for (int i = 0; i < DECK_SIZE - 1; ++i) {
			int j = i + rand_r(&seed) % (DECK_SIZE - i);
			int tmp = deck[i];
			deck[i] = deck[j];
			deck[j] = tmp;
		}

		if (deck[0] % 13 == deck[1] % 13)
			++hits;
	}
	return hits;
}

static void *work(void *_args)
{
```

```
	ThreadArgs *args = (ThreadArgs *)_args;

	args->hits = count_pairs(args->rounds, args->seed);
	return NULL;
}

static unsigned long run_parallel(long n_threads, long rounds)
{
	pthread_t *threads = malloc(n_threads * sizeof(pthread_t));
	ThreadArgs *thread_args = malloc(n_threads * sizeof(ThreadArgs));

	if (threads == NULL || thread_args == NULL)
		die("Ошибка malloc\n");

	for (long i = 0; i < n_threads; ++i) {
		thread_args[i] = (ThreadArgs){
			.rounds = rounds / n_threads + (i < rounds % n_threads),
			.seed = i + 1,
		};

		if (pthread_create(&threads[i], NULL, work, &thread_args[i]) != 0)
			die("Ошибка pthread_create\n");
	}

	unsigned long hits = 0;
	for (long i = 0; i < n_threads; ++i) {
		if (pthread_join(threads[i], NULL) != 0)
			die("Ошибка pthread_join\n");
		hits += thread_args[i].hits;
	}

	free(thread_args);
	free(threads);
	return hits;
}

int main(int argc, char **argv)
{
	long n_threads = 0, rounds = 0;
	int parallel = 0;

	for (int i = 1; i + 1 < argc; i += 2) {
		if (strcmp(argv[i], "-t") == 0) {
			n_threads = atol(argv[i + 1]);
			parallel = 1;
		} else if (strcmp(argv[i], "-r") == 0) {
			rounds = atol(argv[i + 1]);
		}
	}

	if (rounds < 1 || (parallel && n_threads < 1))
		die("Использование: ./lab2 -r <раунды> [-t <потоки>]\n"
		    "без -t работает последовательная версия\n");

	long long start = now_us();
	unsigned long hits = parallel ? run_parallel(n_threads, rounds)
	                              : count_pairs(rounds, 1);
	long long us = now_us() - start;

	/* вероятность с 6 знаками после запятой без чисел с плавающей точкой */
	unsigned long scaled = hits * 1000000 / rounds;

	if (parallel) {
		print(STDOUT_FILENO, "Версия: параллельная, потоков: ");
		print_num(n_threads, 1);
	} else {
```

```
                print(STDOUT_FILENO, "Версия: последовательная");
        }
        print(STDOUT_FILENO, "\nРаундов: ");
        print_num(rounds, 1);
        print(STDOUT_FILENO, ", пар сверху: ");
        print_num(hits, 1);
        print(STDOUT_FILENO, "\nВероятность: ");
        print_num(scaled / 1000000, 1);
        print(STDOUT_FILENO, ".");
        print_num(scaled % 1000000, 6);
        print(STDOUT_FILENO, "\nВремя: ");
        print_num(us / 1000, 1);
        print(STDOUT_FILENO, ".");
        print_num(us % 1000, 3);
        print(STDOUT_FILENO, " мс\n");
        return 0;
}
```

bench.sh (вспомогательный скрипт для замеров)

```
#!/bin/bash
# Собирает таблицу для отчёта: последовательная версия + параллельная для
каждого числа потоков.
# Каждый замер повторяется 3 раза, в таблицу идёт медиана (среднее по величине
из трёх).
# Использование: ./bench.sh <раунды> [числа потоков через пробел]
# Пример:        ./bench.sh 100000000 1 2 4 6 8 12 16 128 1024

R=${1:-100000000}
shift
THREADS=${*:-"1 2 4 6 8 12 16 128 1024"}

measure() {
      for k in 1 2 3; do
            ./lab2 "$@" | awk '/^Время/ { print $2 }'
      done | sort -n | sed -n 2p
}

./lab2 -r "$R" > /dev/null   # прогрев: первый запуск часто медленнее, в
таблицу не идёт
TS=$(measure -r "$R")

echo "Раундов: $R"
echo
echo "| Число потоков | Время исполнения (мс) | Ускорение | Эффективность |"
echo "|---|---|---|---|"
echo "| последовательная | $TS | — | — |"
for t in $THREADS; do
      TP=$(measure -r "$R" -t "$t")
      awk -v t="$t" -v ts="$TS" -v tp="$TP" \
            'BEGIN { s = ts / tp; printf "| %d | %s | %.2f | %.3f |\n", t, tp,
s, s / t }'
done
```

Протокол работы программы

Сборка и проверка корректности

```
$ gcc -Wall -Wextra -O2 -pthread lab2.c -o lab2
$ ./lab2 -r 10000000
Версия: последовательная
Раундов: 10000000, пар сверху: 588715
Вероятность: 0.058871
Время: 2300.014 мс
$ ./lab2 -r 10000000 -t 1
Версия: параллельная, потоков: 1
```

```
Раундов: 10000000, пар сверху: 588715
Вероятность: 0.058871
Время: 2279.745 мс
$ ./lab2 -r 10000000 -t 4
Версия: параллельная, потоков: 4
Раундов: 10000000, пар сверху: 588210
Вероятность: 0.058821
Время: 629.405 мс
```

Программа собирается без предупреждений. Последовательная версия и параллельная с одним потоком дали одинаковое число пар (588715), так как используют одно и то же зерно генератора. Все оценки близки к теоретическому значению 3/51 ≈ 0,058824: например, отклонение 0,000047 при $R = 10^7$ меньше типичной погрешности 0,000074.

Обработка ошибочных ключей

```
$ ./lab2 -r 100 -t 0; echo $?
Использование: ./lab2 -r <раунды> [-t <потоки>]
без -t работает последовательная версия
1
```

Демонстрация количества потоков средствами ОС

```
$ ./lab2 -r 2000000000 -t 4 & sleep 1; ps -o pid,nlwp,cmd -C lab2;
  grep Threads /proc/$(pgrep lab2)/status; kill $(pgrep lab2)
[1] 11523
    PID NLWP CMD
  11523    5 ./lab2 -r 2000000000 -t 4
Threads:        5
```

Утилита ps (столбец NLWP – number of lightweight processes, число потоков процесса) и файл /proc/<PID>/status показывают 5 потоков: 4 рабочих потока, заданных ключом -t 4, и главный поток, который выполняет main и ожидает рабочие потоки в pthread_join.

Результаты замеров

Таблица 1 – Время выполнения, ускорение и эффективность при R = 100 000 000 раундов

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
последовательная	22742,978	—	—
1	23096,617	0,98	0,985
2	12344,330	1,84	0,921
4	6286,488	3,62	0,904
6	4589,420	4,96	0,826
8	3615,189	6,29	0,786
12	2696,592	8,43	0,703
16	2989,864	7,61	0,475
128	2712,001	8,39	0,066
1024	2702,397	8,42	0,008

Таблица 2 – Время выполнения, ускорение и эффективность при R = 10 000 000 раундов

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
последовательная	2358,823	—	—
1	2372,875	0,99	0,994
2	1245,993	1,89	0,947
4	659,759	3,58	0,894
6	464,852	5,07	0,846
8	378,880	6,23	0,778
12	269,697	8,75	0,729
16	317,735	7,42	0,464
128	267,459	8,82	0,069
1024	315,208	7,48	0,007

Таблица 3 – Время выполнения, ускорение и эффективность при R = 1 000 000 раундов

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
последовательная	240,257	—	—
1	238,116	1,01	1,009
2	126,768	1,90	0,948
4	64,521	3,72	0,931
6	50,431	4,76	0,794
8	38,073	6,31	0,789
12	30,618	7,85	0,654
16	40,399	5,95	0,372
128	43,022	5,58	0,044
1024	77,541	3,10	0,003

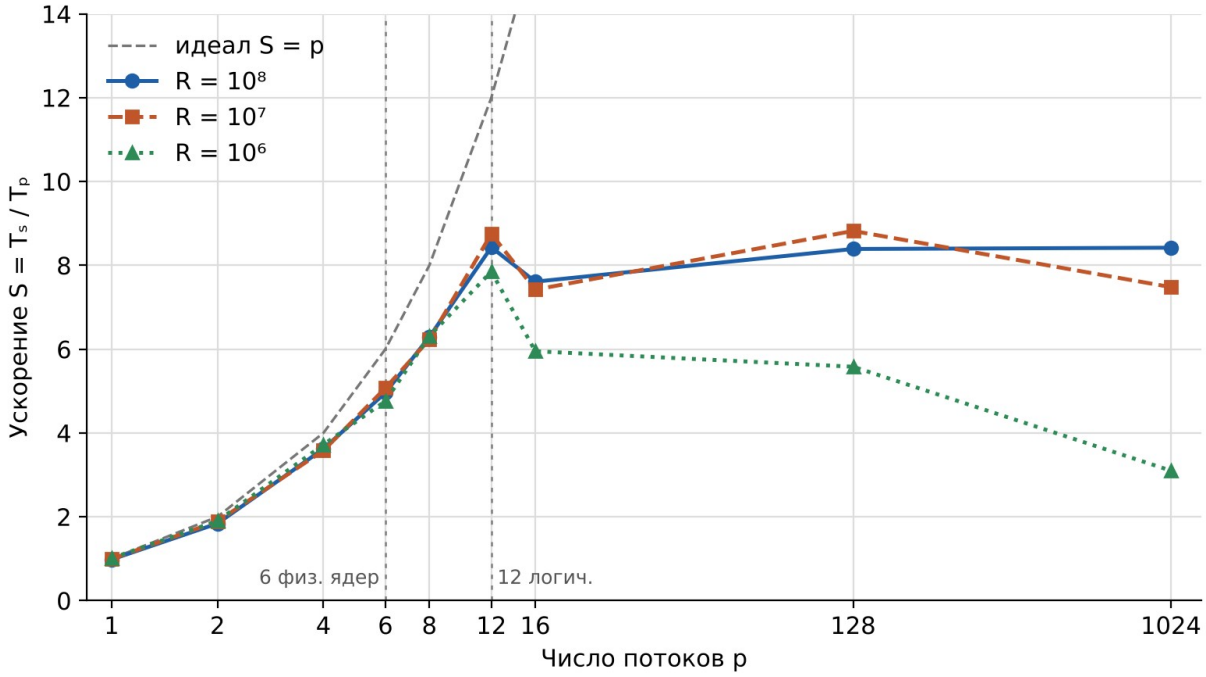


Рисунок 1 – Зависимость ускорения от числа потоков (ось потоков логарифмическая)

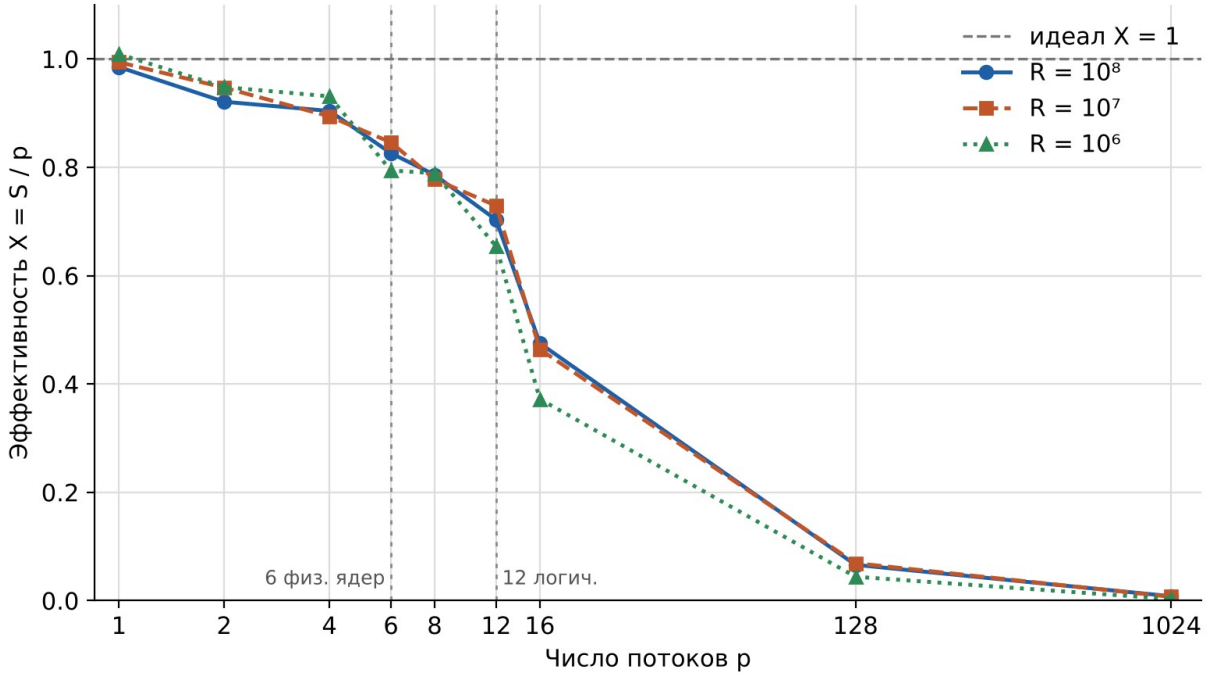


Рисунок 2 – Зависимость эффективности от числа потоков (ось потоков логарифмическая)

Вывод

В ходе лабораторной работы на языке C с использованием POSIX Threads реализованы последовательная и параллельная версии программы, которая методом Монте-Карло оценивает вероятность того, что две верхние карты колоды имеют одинаковое достоинство. Полученная оценка (≈ 0,0588) совпадает с теоретическим значением 3/51 при любом числе потоков. Каждый поток работает только со своими данными, поэтому параллельная версия не требует мьютексов, а максимальное ускорение составило 8,4–8,8 раза при 12 потоках, что равно числу логических процессоров.

Объяснение результатов

- **p = 1:** ускорение 0,98–1,01. Параллельная версия выполняет ту же работу, что и последовательная, плюс создание и ожидание одного потока; расхождение находится в пределах погрешности измерений.
- **p от 2 до 6:** ускорение растёт почти линейно (при $R = 10^8$: 1,84; 3,62; 4,96), эффективность 0,83–0,92. Раунды независимы, общих изменяемых данных и блокировок нет, а последовательная часть программы ничтожна, поэтому закон Амдала почти не ограничивает ускорение. Эффективность ниже 1 – вероятно, потому что при загрузке нескольких ядер процессор ноутбука работает на меньшей тактовой частоте, чем при нагрузке на одно ядро, а также из-за фоновой нагрузки Windows и виртуализации WSL 2.
- **p от 6 до 12:** ускорение продолжает расти (с 4,96 до 8,43 при $R = 10^8$), но медленнее, чем число потоков. Физических ядер 6, а следующие потоки выполняются на вторых логических процессорах тех же ядер (SMT). Два потока на одном ядре делят его вычислительные блоки и вместе дают не двукратный, а примерно 1,7-кратный прирост ($8{,}43 / 4{,}96 \approx 1{,}7$). Эффективность, вычисленная относительно числа потоков, снижается до 0,65–0,73.
- **p = 16:** ускорение ниже, чем при 12 потоках (7,61 против 8,43 при $R = 10^8$). 16 равных частей работы нельзя поровну распределить между 12 логическими процессорами: на четырёх процессорах оказывается по два потока, которые работают вдвое медленнее, а общее время определяется самыми медленными потоками. Планировщик ОС частично перераспределяет потоки между процессорами, но дисбаланс полностью не устраняется; добавляются затраты на переключение контекста.
- **p = 128 и 1024 при $R = 10^8$:** ускорение возвращается к уровню 12 потоков (8,39 и 8,42). Работа разбита на много мелких частей, и планировщик распределяет их по 12 процессорам почти равномерно, а время создания потоков мало по сравнению с 2,7 с вычислений. Ускорения выше, чем при 12 потоках, нет, так как новых аппаратных ресурсов не появляется (псевдопараллелизм), поэтому эффективность падает почти обратно пропорционально p (0,066 и 0,008).
- **Зависимость от объёма работы:** чем меньше R, тем больше роль накладных расходов. При $R = 10^7$ время при 1024 потоках на 45 мс больше, чем при 12 (315 и 270 мс). При $R = 10^6$ создание и ожидание 1024 потоков сравнимо с самой работой: 77,5 мс против 30,6 мс при 12 потоках, ускорение падает до 3,10. Выигрыш от многопоточности тем ближе к максимальному, чем больше объём вычислений, а избыточное число потоков при малом объёме работы приводит к замедлению.
- Оптимальное число потоков для данной задачи равно числу логических процессоров (12). Точность оценки от числа потоков не зависит и определяется только общим числом раундов.

Проблемы, возникшие при выполнении работы

- В одной из первых версий состояние генератора случайных чисел хранилось в общем массиве структур потоков и изменялось на каждом шаге тасовки. Состояния соседних потоков попадали в одну строку кэша процессора (false sharing), и ускорение на двух потоках составляло около 1,07 вместо ожидаемых 2. Проблема устранена: состояние генератора копируется в локальную переменную потока.
- Требование «не использовать стандартную библиотеку» допускало два толкования; после уточнения у преподавателя запрет отнесён только к средствам многопоточности threads.h и <thread>.

- Команда wsl по умолчанию открывала служебный дистрибутив docker-desktop, в котором нет компилятора gcc; проблема решена выбором Ubuntu дистрибутивом по умолчанию.
- Первый запуск в серии замеров выполнялся заметно медленнее последующих, так как процессор ещё не выходил на рабочую частоту, что искажало время последовательной версии и все ускорения; в скрипт замеров добавлен прогревочный запуск.